

Plan for Tackling the Energy AI Hackathon Challenge

Understanding Last Year's Challenge (2025)

Last year's **Energy AI Hackathon 2025** tasked teams with developing a data analytics and machine learning workflow in Python to **predict energy usage (Grid kWh, Diesel gal, CNG MMBTU) for 50 wells**, including providing an **uncertainty model with 100 realizations for each prediction** ¹. The dataset included **1083 wells** with features such as number of stages, clusters, estimated vs. actual stage times, frac fleet, target formation, field area (location indicator), ambient temperature, etc., along with the energy consumed from each source ² ³. Each well had a "Fuel Type" (e.g. Grid, Diesel, Turbine/CNG, or DGB dual-fuel) indicating which energy sources were used. The goal was to forecast **Grid, Diesel, and CNG energy requirements** for new wells and quantify uncertainty by producing 100 plausible outcomes per well ⁴ ⁵.

Key requirements from 2025 problem:

- **Multivariate regression:** predict three energy values (some wells use one or two sources depending on fuel type).
- **Data quality:** handle missing or incomplete data (imputation likely needed) ⁶.
- **Feature engineering:** evaluate many predictors, perform feature selection, transformations, and integrate domain knowledge ⁶.
- **Modeling:** train and tune robust machine learning models for forecasting energy use ⁷.
- **Uncertainty quantification:** provide not just point estimates but an ensemble of 100 predictions per well to model uncertainty ⁴.

This general challenge structure (multivariate, spatio-temporal ML problem with uncertainty) is similar each year ⁸, so we can use last year's problem as a template for 2026.

Solution Workflow Outline

To tackle such a challenge, we can break the work into clear steps. Below is a comprehensive plan, with each step and how to articulate it to Replit's AI coding assistant for implementation. This ensures we cover data understanding, modeling, and uncertainty estimation thoroughly.

1. Data Ingestion and Initial Inspection

First, load the provided datasets (e.g. training data CSV and any separate test data CSV). Confirm the data schema and get an initial sense of the content.

- **Task:** Read the CSV data into a pandas DataFrame. e.g.,

```
pd.read_csv('HackathonData2025.csv')
```

- **Ask Replit:** *“Load the dataset into a pandas DataFrame and display the first few rows and summary info (data types, non-null counts).”*

Check for missing values or obvious data issues. For instance, call `df.info()` and `df.describe()` to see null counts and basic stats.

- **Task:** Identify missing data and basic statistical ranges.
- **Ask Replit:** *“Show how many values are missing in each column and calculate basic statistics (mean, min, max) for numeric features.”*

2. Data Cleaning and Imputation

Handle any missing or anomalous data points. Last year’s data had incomplete entries, so teams needed to **impute missing values using domain knowledge or statistical methods** ⁹. For example, if some wells lack “Ambient Temperature” or have null “# of clusters”, decide how to fill them (mean, median, or a domain-driven value). Ensure to document any assumptions.

- **Task:** Impute or drop missing values appropriately. For numeric fields, one might fill with median or mean; for categorical, perhaps fill with a default or mode. If a feature is largely incomplete or irrelevant, consider dropping it.
- **Ask Replit:** *“Impute missing values in the DataFrame: for numeric columns fill with median (or a domain-appropriate value), and for categorical columns fill with the mode. Then verify that no NaNs remain.”*

Also consider outliers or inconsistent values (e.g., negative numbers where only positives make sense, etc.) and correct or remove them if necessary.

3. Exploratory Data Analysis (EDA)

Perform EDA to understand relationships in the data. This includes:

- **Statistical summaries:** Group by fuel type to see differences (e.g. wells using grid vs diesel might have different # of stages). Check correlations between features and target outputs (e.g. do more stages or longer pump times correlate with higher diesel usage?).
- **Visualizations:** Plot histograms or boxplots of key features (stages, clusters, temperatures) and target variables (Grid kWh, Diesel gal, CNG MMBTU) to understand their distributions. Plot pairplots or scatter plots for features vs. energy consumption. For categorical features (Frac Fleet, Formation, Field Area), perhaps visualize average energy use per category.
- **Task:** Generate summary stats and plots.
- **Ask Replit:** *“For each energy source column (Grid, Diesel, CNG), plot a histogram to see its distribution. Also, plot scatter plots of energy consumed vs. key features (like # of stages, temperature).”*

Look for insights such as: “Ambient Temperature” might affect operations, or “Actual vs Estimated Stage Time” differences could indicate efficiency that affects energy use. If wells are grouped by pad (wells with same name are same pad), perhaps energy usage is similar within pads – we might create a pad-level aggregation if relevant.

Identify which features seem most important. For example, you might notice energy consumption scales with number of stages or clusters. Note these insights for feature engineering.

4. Feature Engineering and Selection

Based on EDA and domain intuition, create or select features that improve the model:

- **Feature transformations:** If “Estimated vs Actual Stage Time” difference seems predictive of extra energy (e.g., delays might cause more diesel usage), create a new feature like $\text{Time_Overrun} = \text{ActualAvgStageTime} - \text{EstAvgStageTime}$. Similarly, combine $\#Stages * \text{ActualStageTime}$ to get total pumping time per well.
- **Encoding categoricals:** Convert categorical features like Frac Fleet, Target Formation, Field Area into numeric form (using one-hot encoding or label encoding). For “Well Name” (pad grouping), you might create a group indicator or use pad-average features if pads have consistent patterns.
- **Handling Fuel Type:** Since a well’s “Fuel Type” dictates which of Grid/Diesel/CNG are consumed, you could treat the prediction problem in segments or add fuel-type-specific features. One approach is building separate models per fuel type category (e.g., one model to predict Grid kWh for grid wells, another for diesel consumption in diesel/DGB wells, etc.). Alternatively, include Fuel Type as an input feature in a single model (with one model predicting all three outputs, though multi-output modeling is more complex).

Given the complexity, a simpler approach is to **build one model for each target energy type**. The Fuel Type feature can still be used to tell the model when a certain energy should be zero (for example, if a well is grid-powered, diesel usage is zero). You can train on all wells for each target, and the model can learn that certain features (fuel type) imply low/zero usage for others.

To decide which features to include, leverage **feature importance techniques**. Last year’s workshop covered SHAP (Shapley values) for feature importance, and teams used **importance models to find effective features** ¹⁰. You could ask the model (like a Random Forest) for feature importances or use a library like SHAP to see which features most influence predictions.

- **Task:** Encode categorical variables, create new engineered features, and then reduce the feature set to the most relevant ones.
- **Ask Replit:** *“Encode categorical features (one-hot or label encoding as appropriate) and add new features: e.g., difference between actual and estimated stage time, total pumping time = $\#Stages * \text{ActualStageTime}$, etc. Then, using a preliminary RandomForest model, output feature importance rankings to identify top features.”*

Based on the importance rankings and domain logic, select a subset of features that will be used in the final model training (this helps reduce noise and dimensionality).

5. Model Selection and Training

With clean, prepared data, choose a machine learning model to predict energy consumption. Considering the hackathon time constraint and dataset size, a **Random Forest Regressor** is a strong choice (it’s robust, handles non-linear relationships, and requires less tuning to get reasonable results). In fact, a successful approach in 2024 was to use RandomForest and optimize it ¹¹. Gradient Boosting (XGBoost or LightGBM)

could also work well. If time permits, you might try multiple models and compare performance (Random Forest, XGBoost, even a simple linear regression as a baseline).

Given there are three targets (Grid, Diesel, CNG), you can either train **three separate regression models**, one for each energy type, or use a multi-output regressor. The simpler route is separate models per target to avoid complicating multi-output handling, especially since each well only has one or two energy types in use (many values are zero). For each model, the target variable is the consumption of that energy source; wells where that source wasn't used will have target value 0, which the model should learn from context (fuel type and related features).

- **Task:** Split the data into training and validation sets (if no separate test labels are provided for practice, use cross-validation on training data). Train a regression model for each target.
- **Ask Replit:** *"Using scikit-learn, train a RandomForestRegressor to predict Grid energy (kWh). Use features X, Y, Z (list the selected features). Do the same to train models for Diesel (gal) and CNG (MMBTU). Use 5-fold cross-validation to evaluate performance (e.g., R^2 or MSE)."*

During training, apply **hyperparameter tuning** to improve performance. For Random Forest, key parameters include number of trees (`n_estimators`), max depth, and min samples splits. Use either a simple grid search or scikit-learn's `RandomizedSearchCV` / `GridSearchCV` on the training set (within the hackathon time, a coarse search is fine).

- **Task:** Optimize model hyperparameters via cross-validation.
- **Ask Replit:** *"Perform a grid search to tune the RandomForest hyperparameters (e.g., `n_estimators`, `max_depth`, `max_features`). Print the best parameters found and the corresponding validation score."*

Track the model's performance (e.g., Mean Squared Error or Mean Absolute Error). Ensure the model isn't overfitting — cross-val helps here. If the dataset is large enough (1083 wells), a 5-fold CV will give a reliable estimate.

According to 2024's post-hackathon summary, their team used RandomForest with tuned hyperparameters and **achieved top accuracy** ¹². So, this approach is validated. If time allows, you might also check residuals or error distribution to ensure the model predictions vs actuals are reasonable (and this residual distribution can inform uncertainty modeling).

6. Uncertainty Modeling (Bootstrap Ensemble)

After obtaining a well-tuned predictive model, we need to generate **uncertainty bounds** for each prediction. The hackathon required **100 realizations per well** to represent the prediction distribution ⁴. A practical way to do this, as taught by Prof. Pyrcz and used by teams, is the **Bootstrap method for uncertainty** ¹¹.

Bootstrap approach: Train an ensemble of models on resampled data and collect their predictions. For example, you can create 100 bootstrap samples of the training set (each sample is a random sample with replacement of the original training data), train a model on each sample, and then predict the target for each test well using each model. This yields 100 predictions for each well, which serves as an empirical distribution of outcomes ¹³. This method assumes that by sampling the data and retraining, we account for variability in model parameters and data.

Concretely, for each energy type model: - Generate 100 bootstrap replicates of the training data. - For each replicate, train a new Random Forest model (it helps that RF already has randomness in sampling and feature selection; using different bootstrap data adds more diversity). - Predict the target for all **test wells** with each model and store the results (you will end up with 100 predicted values for each well's energy consumption).

This approach was highlighted by experts as a powerful method to calculate uncertainty in model predictions ¹⁴. In practice, teams have used it successfully – for example, a hackathon team reported “we developed an uncertainty model with the Bootstrap method to gauge our intervals of confidence” ¹¹.

- **Task:** Implement bootstrap ensemble prediction for uncertainty.
- **Ask Replit:** “Implement a bootstrap-based uncertainty quantification: for the trained RandomForest model, generate 100 bootstrapped training sets and train 100 models. For each model, predict the test set outcomes. Collect these predictions into a 100×N array (where N is number of test wells) for analysis.”

Because training 100 full models can be time-consuming, consider simplifying if needed: e.g., use a smaller subset for each or fewer trees per model to save time. Alternatively, since Random Forest itself is an ensemble, one could also approximate uncertainty by looking at the variance of predictions across trees (though extracting per-tree predictions in sklearn RF is possible via `estimators_`). However, the straightforward way to meet the “100 realizations” requirement is the explicit bootstrap ensemble.

After generating the 100 predictions per well, you can compute statistics like mean and standard deviation for analysis, but ultimately the hackathon expects you to **output all 100 values** for each well and energy type. Ensure you maintain the required format (likely each well-energy combination listed with 100 values).

7. Results Formatting and Visualization

Prepare the output in the specified format (for 2025, the submission was a CSV with each test well's name, fuel type, actual value, and 100 realization columns). For 2026, format might be similar – carefully follow this year's instructions. Use the test DataFrame to retrieve well identifiers and fuel types in the correct order, then attach the 100 predictions.

- **Task:** Format the prediction results.
- **Ask Replit:** “Combine the 100 prediction realizations for each well into a DataFrame (or CSV) with columns: Well Name, Fuel Type, and R_1 ... R_100 for the realizations. Save this to a CSV file.”

Additionally, consider providing some **visualization of uncertainty** for the presentation: for example, plot prediction distributions for a few sample wells (perhaps show a histogram or confidence interval for high vs low usage wells). This can impress judges by demonstrating understanding of the results. If using SHAP or feature importance, a plot of feature importance or SHAP summary could also be useful in the presentation to show which factors drive energy consumption.

8. Testing and Iteration

With the workflow implemented, test it on last year's data to ensure it works end-to-end. For example, you can run the pipeline on the 2025 dataset (which we have ground-truth for in `solution.csv`) to see if the

predictions make sense and perhaps measure the accuracy of your point predictions against the true values (e.g., calculate MSE between the actual fuel values and your prediction means).

- If certain energy types show large errors, you might iterate: e.g., maybe Diesel predictions are poor – perhaps separate the model for DGB diesel vs straight diesel wells, or add interaction features. Use residual analysis to guide improvements.
- Time is limited in a hackathon, so prioritize the most impactful improvements (those suggested by domain insights or large errors).

Throughout, **integrate domain expertise**. For instance, if you know that “Field Area” clusters wells geographically, perhaps group by field to see if different fields have systematically higher or lower energy usage (maybe due to differing geology or practices). Domain insight might also guide imputation (e.g., if a certain formation typically has similar cluster counts, use that info to fill missing values).

Finally, ensure the code is well-organized (the judges even scored “code structure” in the past ¹⁵). Use clear function definitions for repetitive tasks (like training a model, doing bootstrap, etc.) so that it’s easier to adjust or debug during the hackathon.

Using Replit’s AI for Implementation

Leveraging Replit’s AI coding assistant can accelerate writing this code. The key is to **give clear, specific instructions for each step**. Here’s how you might articulate each part of the plan to Replit:

1. **Loading Data:** “Use pandas to read the provided CSV file (e.g. `HackathonData2025.csv`). Then print `df.head()` and `df.info()` to confirm the data loaded correctly.”
2. **Exploring Data:** “Calculate summary statistics of the DataFrame (e.g., use `df.describe()`), and identify any columns with null values (`df.isna().sum()`).”
3. **Cleaning:** “Impute missing values: for numeric columns use median, for categorical use mode. For example, use `df['Column'].fillna(df['Column'].median(), inplace=True)` for numeric columns. Then verify no more nulls with `df.isna().sum()`.”
4. **Feature Engineering:** “Create a new feature for each row: `TimeOverrun = ActualAvgStageTime - EstAvgStageTime`. Also create `TotalPumpingTime = ActualAvgStageTime * NumStages`. Add these as new columns to the DataFrame.” Next, “Convert categorical features (like Fuel Type, Frac Fleet, Field Area) to numeric: use one-hot encoding via `pd.get_dummies` or `LabelEncoder` as needed.”
5. **Feature Selection:** “Use a `RandomForestRegressor` to fit on the data (just a quick fit) and output the feature importances (`model.feature_importances_`). List the top 10 features.” (This can be done after splitting X and y for one of the targets, to get an idea.)
6. **Model Training:** “Split the data into training and validation (for example, 20% for validation). Train a `RandomForestRegressor` to predict `Grid` energy. Evaluate its R^2 or MSE on the validation set.” Repeat for Diesel and CNG.
7. **Hyperparameter Tuning:** “Perform a grid search on the `RandomForest` for `Grid` prediction: vary `n_estimators` (50, 100, 200) and `max_depth` (None, 5, 10), using 5-fold cross-validation to find the best combination. Print the best params and score.” Do similarly for the other models (or assume similar optimal parameters might apply).
8. **Uncertainty via Bootstrap:** “Implement bootstrap uncertainty: for the `Grid` model, loop 100 times, each time sample the training data with replacement (use sklearn’s `resample` or `np.random.choice` on

indices) to create a bootstrap sample. Train a new RandomForest (with the tuned hyperparams) on each sample and predict the Grid energy for all test wells. Collect all 100 predictions for each test well.” Then do the same for Diesel and CNG models.

9. **Output Format:** *“Organize the 100 predictions per well into the required output format. For example, create a DataFrame with columns: Well Name, Fuel Type, Fuel Value (if known for test, or leave blank if unknown in actual submission), and R_1 ... R_100 for predictions. Save to CSV.”*

By giving Replit such step-by-step prompts, you'll get code for each part. You can run and verify each step before moving on (this iterative prompting is key to catching issues early). If Replit's response isn't exactly right, clarify or adjust the prompt (e.g., “No, use median instead of mean for imputation on that column” or “Please include the import statements at the top”).

Throughout the process, keep the **readability** of output in mind. Use comments in code (Replit can add them if you prompt it) to explain each section – helpful for you and to show judges your thought process. Also, version-control your progress (e.g., using Replit's git integration if possible) so nothing breaks as you iterate.

Adapting to the 2026 Hackathon Challenge

When the **2026 hackathon problem** is revealed, quickly identify what's similar or different from 2025:

- **Problem type:** Is it again a prediction of some numeric outcome per well (likely yes)? Note the specific target variable(s) – e.g., it might be a different metric than energy volume, like a rate or an efficiency metric.
- **Features available:** They may provide new features or fewer features. Scan the data description to see what each feature means. Many will likely overlap (stages, clusters, etc., if it's again about drilling/completions), but confirm any new ones and consider their relevance.
- **Output requirements:** Ensure you understand if they still want multiple realizations for uncertainty (very likely, as uncertainty has been a theme). If it's 100 again or some other number, adjust accordingly.

Once you have this information, **apply the same workflow:**

1. **Ingest and inspect** the new data (using the same Pandas steps).
2. **Clean and preprocess** (impute missing values, handle categories) – tailor the imputation to any new feature distributions or domain knowledge for 2026.
3. **EDA** on the new data – look for patterns, outliers specific to the new challenge (e.g., maybe new categorical values, or a wider range of ambient temperatures, etc.).
4. **Feature engineering** – create analogous features (time differences, totals, ratios) if they make sense, or any new combinations relevant to the new target.
5. **Model training** – likely the same algorithm (Random Forest or similar) will be suitable. If the new target is fundamentally different (say classification instead of regression, though unlikely given the hackathon's history), choose the appropriate model type. Otherwise, use the tuned hyperparameters from last year as a starting point (they often remain reasonable). Adjust if needed (e.g., if data size grew, maybe more trees).
6. **Uncertainty** – implement bootstrap or an alternative if introduced. (If organizers hint at another method, you could use that, but bootstrap will generally work regardless.)

7. Output – format according to this year’s spec.

The key is that our plan is modular. For example, if 2026’s challenge is similar in structure to 2025’s (which was similar to 2024’s as we saw), our outline covers the bases. If there are twists (say a time-series component or spatial coordinates given explicitly), we can integrate those: e.g., if a **time series** aspect is present (spatiotemporal data), include time-based features or use sequence models; if **spatial coordinates** given, consider distance-based features or spatial clustering. But the general process of “**understand data → clean → feature engineer → model → uncertainty**” remains consistent.

Finally, remain flexible and communicative with the Replit assistant. If new challenges arise (like a data format issue or an error in modeling), break down the problem and ask Replit for specific help on that issue (e.g., “I’m getting a convergence warning from XGBoost, how can I address it?”). Since hackathons are time-bound, use the AI to quickly try solutions, but also use your judgment to decide when to stick with a simpler working approach versus trying an exotic technique.

By following this plan and clearly conveying each step to Replit, you’ll be able to rapidly prototype a solution for the 2026 challenge. Good luck at the hackathon, and happy coding!

Sources:

- Energy AI Hackathon 2025 problem description and data features ¹ ³ .
- UT PGE Energy AI Hackathon overview (annual spatiotemporal ML challenge) ⁸ .
- Oscar Roman’s summary of their 2024 hackathon solution (data cleaning, feature selection, Random Forest model, and bootstrap for uncertainty) ⁹ ¹² .
- Michael Pyrcz’s guidance on using bootstrap for uncertainty modeling ¹⁴ .

¹ ² ³ ⁴ ⁵ ⁶ ⁷ GitHub - PGEHackathon/data2025

<https://github.com/PGEHackathon/data2025>

⁸ Energy AI Hackathon - Hildebrand Department of Petroleum and Geosystems Engineering | The University of Texas at Austin

<https://www.pge.utexas.edu/energy-ai-hackathon/>

⁹ ¹⁰ ¹¹ ¹² ¹⁵ Last weekend, I had a wonderful (and very daunting) opportunity to take part in the Energy A.I. 2024 4th annual Hackathon hosted by the Hildebrand Department of Petroleum and Geosystems Engineering!... | Oscar Roman

https://www.linkedin.com/posts/oscar-roman-47183921b_last-weekend-i-had-a-wonderful-and-very-activity-7157069753180647424-NOQ-

¹³ ¹⁴ #Bootstrap is a very powerful method in #statistics, #DataAnalytics ...

<https://x.com/GeostatsGuy/status/1213875105568378882>